

Tyee: A Unified, Modular, and Fully-Integrated Configurable Toolkit for Intelligent Physiological Health Care

Tao Zhou*
Hunan University
Changsha, China
zhoutao@hnu.edu.cn

Lingyu Shu*
Hunan University
Changsha, China
shulingyu@hnu.edu.cn

Zixing Zhang†
Hunan University
Changsha, China
zixingzhang@hnu.edu.cn

Jing Han†
University of Cambridge
Cambridge, UK
jh2298@cam.ac.uk

Abstract

Deep learning has shown great promise in physiological signal analysis, yet its progress is hindered by heterogeneous data formats, inconsistent preprocessing strategies, fragmented model pipelines, and non-reproducible experimental setups. To address these limitations, we present Tyee, a unified, modular, and fully-integrated configurable toolkit designed for intelligent physiological healthcare. Tyee introduces three key innovations: (1) a **unified** data interface and **configurable** preprocessing pipeline for 12 kinds of signal modalities; (2) a **modular** and extensible architecture enabling flexible integration and rapid prototyping across tasks; and (3) **end-to-end** workflow configuration, promoting reproducible and scalable experimentation. Tyee demonstrates consistent practical effectiveness and generalizability, outperforming or matching baselines across all evaluated tasks (with state-of-the-art results on 12 of 13 datasets). The Tyee toolkit is released at <https://github.com/SmileHnu/Tyee> and actively maintained.

CCS Concepts

• **Software and its engineering** → **Open source model**; • **Applied computing** → **Life and medical sciences**.

Keywords

Open source, physiological signal, health and well-being

ACM Reference Format:

Tao Zhou, Lingyu Shu, Zixing Zhang, and Jing Han. 2025. Tyee: A Unified, Modular, and Fully-Integrated Configurable Toolkit for Intelligent Physiological Health Care. In *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25)*, October 27–31, 2025, Dublin, Ireland. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3746027.3756868>

1 Introduction

Physiological signals, such as electroencephalography (EEG), electrocardiography (ECG), and electromyography (EMG), are essential

*These authors contributed equally to this research.

†Zixing Zhang and Jing Han are corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MM '25, Dublin, Ireland

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2035-2/2025/10
<https://doi.org/10.1145/3746027.3756868>

to understand physiological and cognitive states [7, 9]. Their growing utilization in intelligent healthcare has driven deep learning methods for physiological signal analysis [1, 3]. In particular, these methods have shown promising performance in critical healthcare tasks from disease prediction to patient monitoring [5, 12].

Recently, several open-source toolkits have been developed to support physiological signal analysis. As summarized in Table 1, these toolkits provide a strong foundation and have contributed significantly to advancing the field. However, despite their utility, researchers and developers (e.g., physiological signal engineers and machine learning practitioners) continue to face *three key challenges* that limit broader adoption. First, *data integration remains a major challenge* due to fragmented data formats and inconsistent preprocessing pipelines across datasets; *a simple, flexible interface* that hides these differences and makes it easier to work with data from many sources in a consistent way is still lacking. Second, *model development is often slowed by rigid, monolithic toolkits* that limit experimentation and make fair comparisons difficult; *a modular framework* that encapsulates core components while enabling independent configuration and substitution remains an unmet need. Third, *inconsistent workflows, hidden defaults, and inflexible code-bases* often lead to inconsistent and unreliable outcomes; *a streamlined system that automates the pipeline*—reducing manual coding while maintaining transparency—remains critically needed.

To address these three challenges, we introduce an open-source *Tyee: a unified, modular, and fully-integrated configurable deep learning Toolkit for physiological health care*. The name Tyee, pronounced like the Chinese term “太医” (meaning *imperial physician*), reflects its mission to bridge intelligent computing with physiological healthcare. As the first toolkit of its kind, Tyee pioneers accessible physiological signal analysis. It is designed to serve both *novice users* requiring ready-to-use pipelines and *advanced researchers and developers* focused on developing, evaluating, or benchmarking custom models across diverse datasets and tasks. To accomplish this, it provides: (i) a unified data interface that abstracts dataset-specific preprocessing through configurable pipelines, enabling seamless integration and reuse; (ii) a modular architecture supporting flexible configuration (models, loss functions, optimizers, and evaluation metrics), enabling rapid prototyping and fair benchmarking; and (iii) holistic workflow configuration that automates the entire experimental process, from data preprocessing to evaluation, boosting reproducibility and scalability with minimal coding.

2 Tyee: Design & Features

In this section, we first present the overall architecture of Tyee, followed by a detailed discussion of its key features, including

Table 1: Comparison of Tyee and several existing toolkits. “Code” = only usable via code, “Code/Config” = both code and configuration files can be used, “DL” = deep learning, ✓ = supported, ✗ = unsupported.

Toolkit	Modality	Preprocessing	DL Support	Modular	Pipeline
MNE-Python [2]	EEG, MEG, ECoG, sEEG, fNIRS, EMG, EOG, ECG	Code	✗	✗	✗
NeuroKit2 [6]	ECG, PPG, EDA, EMG, EOG, Resp, HRV	Code	✗	✓	✗
PyHealth [10]	EHR (ts, codes, tabular)	Code	✓	✓	Code
TorchEEG [13]	EEG only	Code	✓	✓	Code
TorchECG [8]	ECG only	Code	✓	✓	Code
Tyee (Ours)	over 12 physiological signals	Code/Config	✓	✓	Code/Config

the unified data interface, configurable preprocessing, modular component design, and end-to-end configuration.

2.1 Design Principles & Tyee Architecture

Inspired by prior works [4, 11] and to address key limitations in existing physiological signal toolkits, Tyee is developed based on three guiding principles. **Reproducibility** is improved by decomposing the workflow into modular components with encapsulated configurations, enabling transparent editing and reliable evaluation. **Standardization** is met by including frequently used modules with a single unified interface, lowering the barrier for collaborative and interdisciplinary research. **Extensibility** is achieved by encapsulating and integrating new models or algorithms via lightweight coding efforts, supporting flexible and scalable development.

Following these design principles, we present the architecture of Tyee in Figure 1. The system is composed of configurable modules structured within a layered architecture, enabling a flexible yet end-to-end workflow for physiological signal analysis. From a bottom-up perspective, it encapsulates all major processing steps across four layers—*Entity*, *Task*, *Training*, and *Configuration*—with each layer abstracting a specific functionality.

2.2 Unified Data Interface & Preprocessing

This design enables users to standardize diverse datasets and customize preprocessing workflows tailored to specific task requirements. By encapsulating common data operations, such as reading, preprocessing, and storing, into a consistent and extensible workflow, Tyee simplifies handling heterogeneous physiological signals. Specifically, it provides the following core functionalities.

- `read_record()`: this allows users to load physiological signals from various file formats (e.g., .edf, .mat, .cnt, .bdf), and supports easy integration of new datasets through user-defined format-specific parsers.
- `online_transform` and `offline_transform`: two kinds of configurable transform parameters are introduced to accommodate varying preprocessing requirements. This enables task-specific processing of signals and labels via configuration files, respectively. To further streamline development, it also offers a collection of built-in transforms covering common preprocessing routines.
- `__getitem__()`: this provides a unified interface for retrieving preprocessed samples, ensuring consistent and efficient access for various tasks.

2.3 Modular Components

Tyee adopts a layered architecture, including the Entity Layer, Task Layer, Training Layer, and Configuration Layer, as illustrated in Figure 1. Each layer is structured with well-defined interfaces and encapsulates modular components.

Entity Layer. This encapsulates core modeling components, including: datasets, models, criteria, optimizers, and evaluation metrics.

Data Module supports diverse physiological signals via a unified interface and configurable preprocessing. New datasets can be integrated by implementing the `read_record()` method. Each data transform (called Preprocessor) is implemented as a subclass of the `BaseTransform` class and overrides the `transform()` method. Custom preprocessors can be registered and configured through YAML files.

Model Module is built on PyTorch and incorporates a range of deep learning models commonly applied to physiological signal analysis, such as sleep staging, seizure detection, and emotion recognition. New models can be added by implementing standard PyTorch classes (i.e., subclasses of `torch.nn.Module`) using conventional `__init__()` and `forward()` methods.

Optimizer and Other Modules. Additional components such as optimizers, criteria, learning rate schedulers, and evaluators are encapsulated within the Entity Layer. Tyee includes a collection of built-in criteria, learning rate schedulers, and common evaluation metrics to support various training strategies and tasks.

Task Layer. This layer dynamically loads and initializes components such as the model, dataset, criteria, and metrics based on module paths and class names specified in the configuration file. It orchestrates these components to execute training and evaluation workflows. Users can customize behavior by overriding `train_step()` and `valid_step()` methods.

Training Layer. This layer manages the common and end-to-end procedures for training and evaluation. It coordinates task initialization, task execution, checkpointing, and logging. To meet the demands of large-scale computational resources, the Training Layer supports distributed training via standard PyTorch strategies.

Configuration Layer. This layer provides centralized environment settings through YAML configuration files. It governs the setup of datasets, models, tasks, and hyper-parameters. Additionally, Tyee also provides a unified command-line interface to enable fast overrides of configuration parameters.

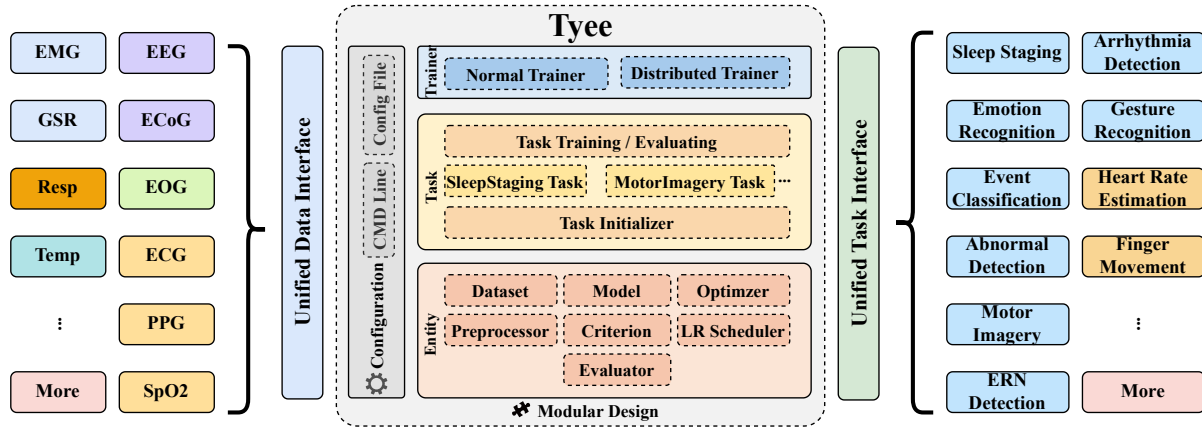


Figure 1: Architecture of the Tyee toolkit. It features modular components and end-to-end configurability, supporting a diverse range of physiological signals and healthcare applications via unified data and task interfaces.

2.4 Configurable Pipeline

Tyee adopts a YAML file-based configuration system that declaratively defines all experiment parameters through structured files. It supports consistent module initialization and management across experiments. Each specific task (as shown in Figure 1 right) is defined within a YAML file, outlining all components of the experiment pipeline in logic sections, from data handling to model architecture and training routines. More specifically, it consists of *common* for global settings, *dataset* for data access, *model* for model details, *optimizer* for learning strategies, *task* for training objective, *trainer* for training-specific configurations, and *distributed* for distributed training setup.

During training, the YAML file is loaded, parsed and passed to the task layer to initialize corresponding components. Users can fully define, reproduce, and adapt experiments by simply modifying configuration files, without changing the underlying code. In addition, Tyee also provides a command-line interface to launch workflows using specified YAML files or runtime overrides.

Complete documentation of all configurable parameters is available in the GitHub repository.

3 Evaluation and Results

Datasets & Models & Metrics. To demonstrate the practical utility and versatility of Tyee, we conducted extensive experiments using a diverse collection of physiological signal datasets. We included comprehensive examples for our evaluation, as detailed in Table 2. In particular, we systematically evaluated the toolkit on 13 public datasets, covering 12 distinct signal types and 11 tasks. The evaluation features state-of-the-art models, including the recent EEGPT. In total, over 4,800 subjects and their 1.2M+ samples were involved, and the supported signal sampling rates range from 32 Hz to over 2 kHz, demonstrating Tyee’s flexibility, with actual capability likely extending beyond these benchmarks.

For *unimodal* tasks, we evaluated eight benchmark datasets. Advanced models from prior studies were reproduced using original implementations or publicly available code, with necessary adaptations for compatibility within Tyee. For *multimodal* tasks, we

utilized five public datasets to assess Tyee’s ability to process and integrate multiple physiological signals. Notably, the SleepEDFx dataset includes two subsets: SleepEDFx-39 and SleepEDFx-158, and here SleepEDFx-39 was used in our experiments. Again, relevant models were re-implemented to ensure compatibility within Tyee. The only exception is the SleepFM model, which was deployed for the sleep staging task on CinC2018 dataset. The original logistic regression classifier was replaced with a linear head to ensure compatibility with the Tyee interface. This modification does not alter the model’s core architecture or performance characteristics but allows for consistent handling of model outputs across tasks.

For the evaluation, in all cases, we followed the original studies’ hyperparameter configurations and reported metrics as applied in their official implementations. This ensured fair and consistent comparisons at the most. In addition, to maintain uniformity in result presentation, we averaged metric values when multiple evaluation metrics were reported for a dataset. As an example, in the case of the SleepEDFx dataset, the original study reported both accuracy and F1-score; hence we reported the average of the two for consistency. Moreover, for the PPG-DaLiA dataset, we further transformed the MAE values using the formula $100/\ln(1 + \text{MAE})$ to facilitate consistent scaling and comparability with other tasks. The full catalog of the explored dataset references and original model implementations can be found in the project’s GitHub repository, due to space limitations.

Results on Unimodal Tasks. Figure 2 presents performance comparisons between Tyee and official baseline models using radar plots for both (a) unimodal and (b) multimodal evaluations. As shown in Figure 2 (a), Tyee demonstrates comparable or superior performance across all eight unimodal tasks, with particularly strong results on the PhysioP300 and BCIC-4 datasets. While conservative model choices may underrepresent its full potential, these results highlight Tyee’s ability to reliably reproduce benchmarks while maintaining robustness and generalizability across diverse physiological signal analysis tasks.

Results on Multimodal Tasks. We observe a similar pattern of performance in the multimodal tasks, as shown in Figure 2 (b). Specifically,

Table 2: Overview of the 13 datasets used to benchmark Tye, covering 12 physiological signal types, 11 baseline models, and 11 different healthcare tasks. Top: eight unimodal datasets. Bottom: five multimodal ones. Statistics include subject count, sampling rate (SR), sample size, and evaluation metrics. *Full dataset references and original model implementations are available on GitHub.*

Dataset	Task	Signals	Model	#Subj.	SR (Hz)	#Samples	Evaluation Metrics
TUEV	Event Classification	EEG	LaBraM	2,383	256	112,491	BA, WF1, κ
TUAB	Abnormal Detection	EEG	LaBraM	288	256	409,455	BA, AUPRC, AUROC
BCIC-2A	Motor Imagery	EEG	EEGConformer	9	250	5,760	Accuracy
BCIC-4	Finger Movement	ECoG	FingerFlex	3	1,000	39,726	Mean CC
KaggleERN	ERN Detection	EEG	EEGPT	26	200	8,840	BA, AUROC, κ
PhysioP300	P300 Recognition	EEG	EEGPT	12	2,048	23,699	BA, AUROC, κ
MIT-BIH	Arrhythmia Detection	ECG	ECGResNet34	40	360	103,979	Accuracy
NinaproDB5	Gesture Recognition	EMG	EMGBench	10	200	39,597	Accuracy
PPG-DaLiA	HR Estimation	PPG, 3D Acc	BeliefPPG	15	64/32	64,607	MAE
SEED-V	Emotion Recognition	EEG, EOG	G2G-ResNet18	20	1,000	148,080	Accuracy
DEAP	Emotion Recognition	GSR, PPG, Resp, Temp	MLSTM-FCN	32	512	25,600	Accuracy
SleepEDFx	Sleep Staging	EEG, EOG	SalientSleepNet	20	100	42,037	Accuracy, F1
CinC2018	Sleep Staging	EEG, EOG, ECG, SaO2, Resp	SleepFM	1,985	200	221,939	F1, AUROC, AUPRC

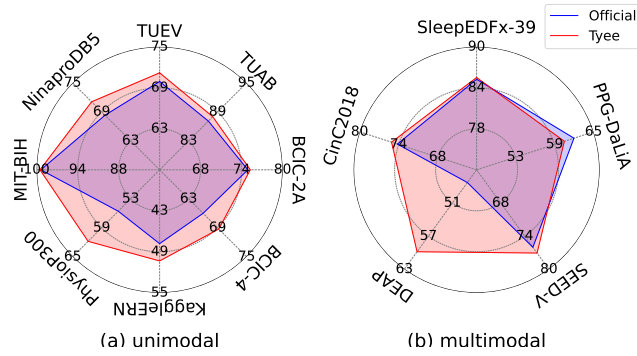


Figure 2: Radar plots comparing Tye (red) vs. baselines (blue) in (a) unimodal and (b) multimodal healthcare tasks. Tye matches or outperforms the baselines on 12 out of 13 datasets, demonstrating reliable and consistent benchmark reproduction.

Tye demonstrates performance on par with the official implementations, achieving slightly better results on three datasets (SEED-V, SleepEDFx, and DEAP), a substantial improvement on DEAP, and a slightly weaker (but not significantly) performance on PPG-DaLiA. Overall, these results highlight Tye’s efficiency and effectiveness in supporting multimodal physiological signal analysis.

4 Conclusion

We present Tye, a user-friendly and comprehensive open-source toolkit for intelligent physiological signal analysis. It is released under the CC BY-NC 4.0 License and available on GitHub. We provide complete source code, detailed documentation, and illustrative examples spanning a wide range of physiological signals and healthcare applications, with easy installation via Conda and Docker. We envision Tye as a foundational platform that accelerates research, fosters collaboration, and streamlines the innovation pipeline from hypothesis to deployment. By reducing redundancy and enhancing reproducibility, Tye aims to facilitate the development of scalable

intelligent health solutions with potential benefits for patient care and public health.

Acknowledgments

This work was supported by the Guangdong Basic and Applied Basic Research Foundation under Grant No. 2024A151010112, and the Changsha Science and Technology Bureau Foundation under Grant No. kq2402082.

References

- [1] Xun Chen, Chang Li, Aiping Liu, et al. 2022. Toward open-world electroencephalogram decoding via deep learning: A comprehensive survey. *IEEE Signal Processing Magazine* 39, 2 (2022), 117–134.
- [2] Alexandre Gramfort, Martin Luessi, Eric Larson, et al. 2013. MEG and EEG Data Analysis with MNE-Python. *Frontiers in Neuroscience* 7, 267 (2013), 1–13.
- [3] Jing Han, Zixing Zhang, Cecilia Mascolo, et al. 2021. Deep Learning for Mobile Mental Health: Challenges and recent advances. *IEEE Signal Processing Magazine* 38, 6 (2021), 96–105.
- [4] Daniel Jarrett, Jinsung Yoon, Ioana Bica, et al. 2021. Clairvoyance: A Pipeline Toolkit for Medical Time Series. In *Proc. International Conference on Learning Representations*. 32 pages.
- [5] Shuo Ma, Yingwei Zhang, Qiqi Zhang, et al. 2024. SleepMG: Multimodal generalizable sleep staging with inter-modal balance of classification and domain discrimination. In *Proc. ACM International Conference on Multimedia*. 4004–4013.
- [6] Dominique Makowski, Tam Pham, Zen J Lau, et al. 2021. NeuroKit2: A Python toolbox for neurophysiological signal processing. *Behavior Research Methods* (2021), 1–8.
- [7] Liqiang Nie, Luming Zhang, Yi Yang, et al. 2015. Beyond doctors: Future health prediction from multimedia and multimodal observations. In *Proc. ACM International Conference on Multimedia*. 591–600.
- [8] Hao Wen and Jingsu Kang. 2022. A Novel Deep Learning Package for Electrocardiography Research. *Physiological Measurement* 43, 11 (11 2022), 115006.
- [9] Gareth J Williams, Abdulaziz Al-Baraikhan, Frank E Rademakers, et al. 2023. Wearable technology and the cardiovascular system: the future of patient assessment. *The Lancet Digital Health* 5, 7 (2023), e467–e476.
- [10] Chaoqi Yang, Zhenbang Wu, Patrick Jiang, et al. 2023. PyHealth: A Deep Learning Toolkit for Healthcare Predictive Modeling. In *Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 5788–5789.
- [11] Shu-wen Yang, Po-Han Chi, Yung-Sung Chuang, et al. 2021. SUPERB: Speech processing Universal PERFORMANCE Benchmark. In *Proc. INTERSPEECH*. 1194–1198.
- [12] Jae-Young Yoo, Seyong Oh, Wissam Shalish, et al. 2023. Wireless broadband acousto-mechanical sensing system for continuous physiological monitoring. *Nature Medicine* 29, 12 (2023), 3137–3148.
- [13] Zhi Zhang, Sheng-hua Zhong, and Yan Liu. 2024. TorchEEGEMO: A deep learning toolbox towards EEG-based emotion recognition. *Expert Systems with Applications* 249 (2024), 123550.